UNIP UNIVERSIDADE PAULISTA	ICET – ANÁLISE E DESENVOLVIMENTO DE SISTEMAS TECNOLOGIA DA INFORMAÇÃO	

Trabalho 1
Programação Aplicada em .NET

Prof. Me. Lucas Teodoro dos Santos

Instruções

- Os exercícios deverão ser desenvolvidos em aplicação do tipo Console;
- O trabalho é **individual**;
- **A entrega deverá ser realizada em um repositório no GitHub**;

  No mesmo repositório, poderão ser criadas três pastas, sendo cada uma referente a um exercício.
- **Data de entrega: 30/09/2026**.

Questão 1 – Pattern Matching: Empréstimos em uma Biblioteca

Crie uma aplicação do tipo Console para simular a verificação de empréstimos em uma biblioteca.

A biblioteca precisa verificar se um usuário pode realizar empréstimos de livros de acordo com seu tipo e a quantidade de empréstimos ativos.

Crie uma classe base chamada `UsuarioBiblioteca`, contendo as propriedades:

- `Nome`;
- `QuantidadeEmprestimosAtivos`.

Crie também as seguintes classes derivadas:

- `Aluno`, com a propriedade `Matricula`;
- `Professor`, com a propriedade `Departamento`;
- `Visitante`, com a propriedade `Documento`.

Implemente o método abaixo no arquivo `Program.cs`, utilizando `switch expression` e Pattern Matching:

```
1       public static string VerificarEmprestimo(Object obj)
2       {
3           // Implementar
4       }
```

O método deverá retornar:

- `"Usuário inválido"` caso o objeto seja `null`;
- `"Empréstimo autorizado para aluno"` para um objeto do tipo `Aluno` com menos de três empréstimos ativos;
- `"Limite de empréstimos atingido para aluno"` para um objeto do tipo `Aluno` com três ou mais empréstimos ativos;
- `"Empréstimo autorizado para professor"` para um objeto do tipo `Professor` com menos de cinco empréstimos ativos;
- `"Limite de empréstimos atingido para professor"` para um objeto do tipo `Professor` com cinco ou mais empréstimos ativos;
- `"Visitantes não podem realizar empréstimos"` para um objeto do tipo `Visitante`;
- `"Usuário não classificado"` para os demais casos.

O método deverá utilizar:

- Type Pattern;
- Property Pattern;
- Padrões relacionais;
- `switch expression`;
- Padrões `null` e `_`.

No método `Main()`, crie objetos de cada tipo de usuário e chame o método `VerificarEmprestimo()` para apresentar no console o resultado retornado para cada objeto.

Questão 2 – Reflection Aberta e Controlada

Crie uma aplicação do tipo Console para exibir informações de objetos utilizando Reflection.

Crie um Attribute personalizado chamado `ExibirAttribute`. Esse Attribute deverá permitir sua utilização somente em propriedades.

```
        [AttributeUsage(AttributeTargets.Property)]
        public class ExibirAttribute : Attribute
        {
        }
```

Crie uma classe chamada `Equipamento`, contendo as seguintes propriedades:

- `Id`;
- `Nome`;
- `Fabricante`;
- `NumeroSerie`;
- `Valor`;
- `Localizacao`.

Marque com o Attribute `[Exibir]` apenas as propriedades `Nome`, `Fabricante`, `Valor` e `Localizacao`.

No arquivo `Program.cs`, implemente os dois métodos abaixo:

```
        public static void ExibirDadosAberto(object objeto)
        {
            // Implementar
        }

        public static void ExibirDadosControlado(object objeto)
        {
            // Implementar
        }
```

O método `ExibirDadosAberto()` deverá utilizar Reflection para exibir o nome e o valor de todas as propriedades públicas do objeto recebido.

O método `ExibirDadosControlado()` deverá utilizar Reflection para exibir somente o nome e o valor das propriedades que estiverem marcadas com o Attribute `[Exibir]`.

Para implementar os métodos, utilize os recursos de Reflection apresentados em aula, como:

- `GetType()`;
- `GetProperties()`;
- `GetValue()`;
- `GetCustomAttribute()`.

No método `Main()`, crie um objeto da classe `Equipamento` e chame os dois métodos, apresentando no console o resultado da Reflection aberta e da Reflection controlada.

Questão 3 – DTO: Reserva de Hotel

Crie uma aplicação do tipo Console para gerar um relatório de uma reserva de hotel.

Crie uma classe chamada `Reserva`, representando a entidade completa do sistema, com as seguintes propriedades:

- `Id`;
- `NomeHospede`;
- `NumeroQuarto`;
- `QuantidadeDiarias`;
- `ValorDiaria`;
- `StatusInterno`;
- `ObservacaoInterna`.

Em seguida, crie um DTO de saída utilizando `record`, chamado `RelatorioReservaDto`. Esse DTO deverá conter somente as informações que serão apresentadas no relatório:

- `NomeHospede`;
- `NumeroQuarto`;
- `QuantidadeDiarias`;
- `ValorTotal`;
- `Situacao`.

No arquivo `Program.cs`, implemente o método `Mapear()`, responsável por receber um objeto do tipo `Reserva` e retornar um objeto do tipo `RelatorioReservaDto`.

```
1       public static RelatorioReservaDto Mapear(Reserva reserva)
2       {
3           // Implementar
4       }
```

O valor total deverá ser calculado multiplicando a quantidade de diárias pelo valor da diária. A situação do relatório deverá ser definida como `"Reserva confirmada"`.

O DTO de saída não deverá expor as informações `Id`, `ValorDiaria`, `StatusInterno` e `ObservacaoInterna`.

Por fim, implemente o método abaixo. Ele deverá receber um objeto do tipo `RelatorioReservaDto` e exibir, no console, todas as suas informações.

```
1       public static void ExibirRelatorio(RelatorioReservaDto relatorio)
2       {
3           // Implementar
4       }
```

No método `Main()`, crie um objeto do tipo `Reserva`, chame o método `Mapear()` para gerar o DTO de saída e, por fim, chame o método `ExibirRelatorio()`.